

CS117: Internet-scale Distributed Systems

Instructor: Noah Mendelsohn

TAs: Hamza Khalid, Owen Thomas, Charlotte Versavel

Semester: Fall 2025

Email: ta117@cs.tufts.edu

[Piazza Q&A and updates](#)

Course Description (CS117)

This course examines the **design decisions behind the World Wide Web** and compares them with traditional distributed system paradigms. It covers:

- Distributed objects, pub/sub, queuing, client/server
- Global naming, location-independence, caching, and extensibility
- Declarative vs. procedural languages (e.g. HTML vs. RPC)
- Design principles like Metcalfe's law, Postel's law, and layering
- Formats and protocols: HTML/XML/JSON vs. RPC

 **Goal:** Develop systems-level thinking that applies to any complex software system, not just web services.

Enrollment Notes (CS117)

- **Instructor permission required.**
 - Students must submit an online application in advance to be considered.
 - Not open via SIS registration without professor approval.
 - Course is often **over-subscribed**.
-

Books & Resources (CS117)

- No required textbooks.
 - Recommended: Safari Books Online (Tufts library access).
 - Readings and papers provided by the instructor.
-

Attendance & Participation (CS117)

- **Mandatory in-person attendance.**

- Participation in class discussions is **graded**.
 - Recordings are only available **with approved excuse** (e.g. illness, interviews).
 - Classroom sessions are **not livestreamed**.
-

More information (e.g. grading structure) may be present in other pages, but this summarizes the accessible core structure for the course.

CS131: Artificial Intelligence

Course Type: In-person and online versions

Implementation Language: Python (C++ with instructor approval)

Course Goals (CS131)

By the end of the course, students will be able to:

1. Identify classical and modern AI paradigms and how they relate.
 2. Analyze the structure of problems and choose appropriate AI paradigms.
 3. Implement a variety of AI algorithms from both classical and modern approaches.
-

Assignments & Assessments (CS131)

- **6 assignments** throughout the semester.
 - Manual grading, submitted as `.py` or `.cpp` (with CMake config)
 - 10% late penalty per 24h, max 1 week
 - **3 × 4-day extensions** allowed per semester
 - **3 exams:** mix of multiple choice, open-ended, and calculations
 - **Reading homework** from textbook may be assigned
 - **Attendance (in-person only):**
 - Based on short questionnaires (no grade impact)
 - Max 2 missed allowed before impact
-

Grading Breakdown (CS131)

Component	In-Person	Online
Coding Assignments	45%	50%
Exams	50%	50%
Attendance	5%	—

- **Minimum grade for passing components: 70 (C-)**
- No curve. Standard grade scale will apply.

Late Policy & Notes

- Homework must be original and AI tools are not allowed
- Grade disputes must be submitted within **1 week** of return
- Failure in either component (assignments or exams) results in failing the course

Academic Honesty

Collaboration is encouraged **outside the assignment itself**, but all submitted code must be your own. Use of generative AI tools is prohibited. Plagiarism will be reported to the Dean.

Accessibility & Support

Students needing accommodations should contact ACCESSIBILITY@TUFTS.EDU or call 617-627-4539. Early notice is essential to ensure timely support.

CS116: Introduction to Security

Instructor: Ming Chow

Sections:

Email: ming.chow@tufts.edu

Course Description (CS116)

A practical, comprehensive course on cybersecurity principles and tools. Topics include:

- Attacks & defense on networks and web systems
- Cryptography, reverse engineering, malware, forensics

- Static and dynamic code analysis
- Capture The Flag (CTF)–style labs and contests

Hands-on work emphasized.

Prerequisites (CS116)

- CS 15 or equivalent (data structures)
- **CS 30 or CS 40 recommended**, but **not required**
- Clarified that CS 40 is **not a hard prerequisite**

Required Tools & Setup

- Modern browser (Chrome, Firefox, Safari, etc.)
- Unix/Linux CLI (Mac/Linux, WSL, or Docker OK)
- Required security tools:
 - Wireshark, Nmap, Netcat, Python, Scapy
 - John the Ripper, Burp Suite, Apktool + Java

 **No use of Kali VM** — due to accessibility, hardware, and performance concerns

Assessment

Component	Weight
Labs	80%
Quizzes (x2)	20%

- All labs submitted via Canvas (Tufts UTLN required)
- Quizzes administered via Canvas
- Discussions and announcements on Piazza

Lab Workload Summary (CS116)

Lab #	Title	Length / Notes
1	Command Line Basics	1-3+ hours
2	Packet Sleuth	1-3 hours
3	Scanning & Reconnaissance	0.5-2+ hours (<i>target involved, not public</i>)
4	Python + Incident Alarm	3+ hrs to very long
5	Password Cracking Contest	Crack → get A
6	XSS Game	Medium
7	Gain Website Access	Very short (<i>live target</i>)
8	Team CTF Game (1 week)	<i>not public</i>
9	Technical Risk Analysis	Short-medium
10	Android Malware Analysis	Short-medium

Notes

- No TAs – instructor directly manages grading and support
- Labs involving real targets are not made public
- Yes, videos are recorded for Twitch and Zoom sessions
- Course satisfies MS in **Cybersecurity & Public Policy** and **Software Systems Development**

Platform Policy

- Piazza used heavily for Q&A and community interaction
- Separate Canvas used for submission — internal access only
- Public-facing course website exists to support outreach, learning transparency, and backup access

Certification & Professional Use

Students may request a **reimbursement letter or certificate** for employers or professional development use. Contact the instructor for arrangements.

CS136: Statistical Pattern Recognition

Instructor: Mike Hughes

TA: Xiaohui Chen

[Course Website](#)

Course Description (CS136)

This course provides a foundation in **probabilistic machine learning**, with a focus on:

- Discrete and continuous probabilistic models
 - Inference techniques and uncertainty quantification
 - Linear and generalized linear models
 - Mixture models and hidden Markov models
 - Applications of Bayesian modeling
-

Learning Objectives (CS136)

After the course, students will be able to:

- Mathematically formulate probabilistic models
 - Select and apply appropriate models for real datasets
 - Analyze accuracy and stability of inference algorithms
 - Understand scalability and runtime tradeoffs of ML methods
-

Prerequisites (CS136)

Students should have prior experience in:

- **Probability theory** (e.g., PDF vs. CDF)
 - **Linear algebra** (e.g., least-squares regression, matrix ops)
 - **Gradient-based optimization**
 - **Python ML tools:** numpy, matplotlib, scikit-learn
 - Ideally completed: **CS135, EE104**
-

Textbook (CS136)

- **Pattern Recognition and Machine Learning**, Christopher M. Bishop
[Free PDF](#)

Coursework Structure (CS136)

- 5 units, each with:
 - 1 written homework (LaTeX typeset math report)
 - 1 coding practical (Python code + analysis)
 - In-class quiz at the end of each unit (~15 minutes)
 - 1 final **project** in the last month (applied modeling in pairs)
-

Grading Breakdown (CS136)

Component	Weight
Homeworks (HW0–HW5)	25%
Coding Practicals	22%
Unit Quizzes (×5)	36%
Final Project	15%
Participation	2%

-  Quizzes: One per unit, lowest dropped
 -  Project includes intermediate checkpoint reports
-

Final Letter Grade Scale (CS136)

Grade	Range
A	0.94–1.00
A–	0.90–0.94
B+	0.87–0.90
B	0.83–0.87
B–	0.80–0.83
C+	0.77–0.80
C	0.73–0.77
C–	0.70–0.73
D	0.63–0.70
F	< 0.60

Note: No rounding up. A 0.8299 is still a B–.

Weekly Time Commitment (CS136)

Approx. **10 hours/week** including:

- Pre-class readings & videos (3.5 hr)
 - Lectures (2 hr)
 - Homework or coding practical (~4 hr)
 - Quiz prep (~0.5 hr)
-

CS175: Graphics (Fall 2024)

Instructor: Remco Chang

Graduate TA: Darby Huye

Course Description (CS175)

This course explores core fundamentals of computer graphics, including:

- 3D rendering via ray casting and ray tracing

- Viewing transformations and shape representation
- Intro to modeling, computer animation, and OpenGL
- Emphasis on C/C++ and strong linear algebra understanding

Prerequisites:

- CS 40 (Machine Structure and Assembly-Language Programming)
- Calc 2
- Background in Linear Algebra (recommended)

[Piazza Link](#)



Textbook (CS175)

Fundamentals of Computer Graphics by Shirley and Marschner



Assignments & Labs (CS175)

- **Assignments (5 total):** 11% each
 - 2%: Written algorithm
 - 9%: Code implementation
 - **Final Project:** 21%
 - **In-Class Labs (8 total):** 3% each
 - Total: 24%
 - Labs must be completed before the next session
 - **Lab 0 not graded but required**
-



Grading Breakdown (CS175)

Component	Weight
Assignment 1	11%
Assignment 2	11%
Assignment 3	11%
Assignment 4	11%
Assignment 5	11%
Final Project	21%
In-Class Labs (×8)	24%
Total	100%

Late Policy (CS175)

- **Assignments:**
 - Due Mondays at 11:59 PM
 - No credit for late work unless written approval obtained
 - Some parts due Friday before (e.g., algorithm worksheet)
- **Labs:**
 - No late submission allowed
 - Checked in class by TA/instructor
- **Final Project:**
 - **Must work by demo day** — otherwise **grade is 0**

Schedule Highlights

- Topics include: OpenGL, Shaders, Ray Tracing, Scene Graphs, and Animation
- Final Presentation: **Dec 17, 2024 (12–2 PM)**

CS160: Introduction to Algorithms

Instructor: Karen Edwards

Course Description (CS160)

This course provides a comprehensive introduction to the design and analysis of algorithms. Topics follow the structure of the CLRS (Cormen et al.) textbook and include:

- Sorting and searching
 - Recursion and recurrence relations
 - Graph algorithms
 - Greedy algorithms and divide-and-conquer
 - Dynamic programming
 - NP-completeness and complexity classes
-

Textbooks & References (CS160)

- **Primary:** *Introduction to Algorithms* (4th ed.) by Cormen, Leiserson, Rivest, and Stein [CLRS]
 - Alternative references include:
 - *Algorithm Design* – Kleinberg & Tardos
 - *Algorithms* – Dasgupta, Papadimitriou, and Vazirani
 - *Algorithm Design Manual* – Skiena
 - *Computer Algorithms* – Baase & Van Gelder
-

Prerequisites (CS160)

- CS 15 and (CS/MATH 61 or MATH 65)
 - Familiarity with:
 - Logarithms
 - Summation notation and geometric series
-

Tools & Platforms (CS160)

- **Piazza:** Discussion, announcements, and handouts
 - **Gradescope:** Homework submission and grading
 - **Canvas:** Video lecture backups (if available)
-

Expected Work (CS160)

- **Homework:** Weekly, due Tuesdays at 11:59 PM
 - **Participation:** Includes lecture quizzes and recitation attendance (2 absences dropped)
 - **Exams:**
 - Midterms: In-class (Feb 19, Mar 26)
 - Final Exam: May 7, 7–9 PM
-

Grading & Evaluation (CS160)

Your final grade will be the **maximum** of the following two weighted formulas:

Option 1

25% Homework

- 5% Participation
- $2 \times 20\%$ Midterm Exams
- 30% Final Exam

Option 2

25% Homework

- 5% Participation
 - $2 \times 15\%$ Midterm Exams
 - 40% Final Exam
-

Instructional Support

- TAs, CAs, Teaching Fellows with regular office hours
 - Office hour schedules and announcements are on Piazza
-

CS121: Software Engineering

Course Description (CS121)

This course explores core principles for building large-scale software systems, focusing on the **coding side** of software engineering. Topics include:

- Abstraction, modularity, architecture

- Specification and design patterns (including concurrency)
- Refactoring, debugging, and testing
- Program verification and synthesis
- Security and the scientific method of debugging

Java is the primary programming language used.

Learning Topics (CS121)

- Java: classes, objects, inheritance, interfaces, generics
 - ADTs, information hiding
 - Software architecture
 - Object-oriented refactoring and testing
 - ACM Code of Ethics
 - Guest lectures on garbage collection and program verification
-

Prerequisites (CS121)

- COMP 40, graduate standing, or instructor permission
-

Projects (CS121)

- **Project 1:** Java ADTs and Adapter Pattern
- **Project 2:** Design Patterns in Java
- **Project 3:** Unit Testing Framework
- **Project 4:** "Java on Rails" application

Projects must be completed individually and submitted electronically.

Grading & Evaluation (CS121)

Component	Weight
Projects/Homework	50%
Readings	9%
Midterm	20%
Final	20%
Meet Your Professor	1%

- **Grading Curve:** Final grades may be curved based on total numeric scores.
 - **Project Requirement:** Failure to make good-faith attempts on **all projects** may result in course failure regardless of scores.
-

Academic Integrity

- You may discuss general programming syntax and Java features
 - **All design and code must be your own work**
 - Implementation help must come from course staff only
-

CS115: Database Systems

Instructor: Dave Lillethun, Ph.D.

Course Description (CS115)

This course covers the fundamental concepts of modern database management systems (DBMS). Topics include:

- Data models: relational, object-oriented, and NoSQL
 - SQL language (SELECT, INSERT, UPDATE, DELETE)
 - Index structures, concurrency, recovery, and query processing
 - Schema design, normalization (3NF, BCNF, 4NF)
 - Working with unstructured/semi-structured/scientific data
 - Choosing the appropriate DB system for a given application
-

Learning Outcomes (CS115)

Students will be able to:

- Explain and compare relational and NoSQL DB models (ACID, CAP, scaling)
 - Write SQL queries and NoSQL operations effectively
 - Design schemas from ER/UML diagrams
 - Enforce constraints using SQL features like triggers/rules
 - Analyze and transform schemas into normal forms
 - Apply DB design patterns and select appropriate DB tech
-

Prerequisites (CS115)

- Understanding of object-oriented programming concepts and common data structures (trees, interfaces, etc.)
-

Required Materials (CS115)

- **Textbook:** *Database Systems: The Complete Book* (2nd Ed) by Garcia-Molina, Ullman, Widom
ISBN-13: 9780131873254
(Any format: hardcover, softcover, eBook, new/used)
 - Laptop capable of running DB software or a virtual machine (Windows, Mac, or Linux)
-

Grading & Evaluation (CS115)

This course uses a mastery-based evaluation system. Each test, quiz, or exam question is rated as:

-  Exceeds Expectations
-  Meets Expectations
-  Needs Improvement
-  Not Assessable

Final Letter Grade Criteria

Grade	Minimum Requirements
A	$\geq 85\%$ Meets on tests, $\geq 30\%$ Exceeds $\geq 85\%$ Meets on quizzes/exams, $\geq 30\%$ Exceeds
B	$\geq 70\%$ Meets on tests, $\geq 10\%$ Exceeds $\geq 70\%$ Meets on quizzes/exams, $\geq 10\%$ Exceeds
C	$\geq 55\%$ Meets on tests and quizzes/exams
D	$\geq 50\%$ Meets on tests and quizzes/exams

+ / - Adjustments

- “+”: Earned by exceeding Exceeds expectations by 10% more than required in both categories
- “-”: Assigned if performance barely meets thresholds without exceeding

Notes

- Assessments are **take-home, open-note, non-collaborative**
- You may use the textbook, your notes, and online materials (no asking or posting)
- All answers must be in your own words

CS201 / DHP D291: Cyber for Future Policymakers

Instructor: Dave Lillethun, Ph.D.

TA (Friday sessions): Laurin Weissinger

Sections:

- CS12 (Undergrad)
- CS201 (Grad CS)
- DHP D291 (Fletcher School)

Course Description (CS201 / DHP D291)

Intro to cyber technologies with a focus on policy relevance. Topics include:

- Internet Architecture, WWW, Cloud Computing
- Open Source, Cryptography, Cybersecurity & Privacy
- Identity Management, AI & ML, Quantum Computing
- Designed for students in policy, IR, and non-CS backgrounds

- Includes **hands-on labs**
-

Learning Outcomes (CS201 / DHP D291)

- Explain technologies to non-technical audiences
 - Write policy-oriented technical briefings
 - Deliver oral presentations on tech topics
 - (Grad sections): Analyze cyber-related policymaking structures
-

Prerequisites (CS201 / DHP D291)

- One programming course (Python/C++/Java with algorithm/data structure exposure; **not JavaScript**)
 - Cannot be taken concurrently
-

Required Materials (CS201 / DHP D291)

- **Textbook:** *Understanding the Digital World*, Brian Kernighan, 2nd Ed.
 - **Software & Tools:**
 - Wireshark
 - 2+ browsers (Chrome, Firefox, etc.)
 - Command-line terminal (e.g., Mac/Linux built-in or Windows WSL)
-

Grading & Evaluation (CS201 / DHP D291)

Your final grade is based on how many assignments in each category achieve “**Meets Expectations**” or higher. Graduate sections (CS201/DHP D291) have the following expectations:

Component	Requirement Summary
Labs (x4)	Must meet expectations to be counted
Briefing Papers (x2)	≥1 Meets for B; both Meets for A
Final Project (Paper + Presentation)	Treated as a combined "Project" grade
Substitution Rule	One briefing paper may substitute for project paper if the latter falls short
+ / - Adjustment	2× "Exceeds" → grade becomes "+"; 0× "Exceeds" → grade becomes "-"
No Final Exam	Replaced by Final Presentation during exam week

Resubmissions:

- Each student gets **8 total attempts** to resubmit any assignment
- Each resubmission costs 1 attempt
- Submitting clearly incomplete work to exploit this policy is not allowed

Registration Guidance (CS201 / DHP D291)

- Undergrads: **CS 12 only**
- Fletcher students: **DHP D291**
- Other grads: **CS 201**
- CSPP students: Choose CS 201 or DHP D291